

Contents

1 Introdução

2 Estruturas de Dados

- 2.1 Dsu
- 2.2 Segtree

3 Grafos

- 3.1 Toposort

4 Teoria e Apêndices

- 4.1 Permutations
- 4.2 Lis

1 Introdução

```
1 #include <bits/stdc++.h>
1 using namespace std;
1 #define fastio ios_base::sync_with_stdio(0); cin.tie(0)
1 #define all(a) a.begin(), a.end()
2 #define rall(a) a.rbegin(), a.rend()
2 #define sz(a) (int) (a).size()
2 #define ll long long
2 #define ld long double
2
2 void dbg_out(string s) { cerr << endl; }
2 template<typename H, typename... T>
2 void dbg_out(string s, H h, T... t){
2     do{ cerr << s[0]; s = s.substr(1);
2         } while (sz(s) && s[0] != ',');
2     cerr << " = " << h;
2     dbg_out(s, t...);
2 }
2
2 #ifdef DEBUG
2 #define dbg(...) dbg_out(__VA_ARGS__, __VA_ARGS__)
2 #else
2 #define dbg(...) 42
2 #endif
2
2 void solution(){
2     int n;
2     cin >> n;
2     for (int i=0; i<n; i++){
2         }
2 }
2
2 signed main(){
2     fastio;
2     int cases=1;
2     //cin >> cases;
2     for (int i=0; i<cases; i++){
2         solution();
2     }
2 }
```

2 Estruturas de Dados

2.1 Dsu

```
6a9 class DSU{
673 public:
1a8     int n;
f92     vector<int> p, r, s;
ea1     DSU(int nodes): p (nodes), r (nodes, 1), s (nodes,
1){
b67         n = nodes;
9e4         for (int i=0; i<n; i++) p[i] = i;
dc3     }
/*
ac5     ll find(ll x){
eab         return (p[x] == x? x : p[x] = find(p[x]));
07f     }
c4c     */
57b     int find(int x){
42e         int root=x;
39f         while (root != p[root]) root = p[root];
133         int aux;
016         while (x != p[x]){
615             aux = p[x];
88a             p[x] = root;
835             x = aux;
8db         }
ea5         return x;
eb6     }
2e7     bool unite(int x, int y){
f08         x = find(x), y = find(y);
e65         if (x == y) return false;
7ce         if (r[x] > r[y]) swap(x, y);
b6a         if (r[x] == r[y]) r[y]++;
d16         s[y] += s[x];
ae9         p[x] = y;
15a         n--;
8a6         return true;
c37     }
082 };
```

2.2 Segtree

```
853 template <class T> class Segtree{
673 public:
1a8     int n;
53f     T neutral;
22c     vector<T> t;
8c2     Segtree(int size, T neutral): neutral(neutral){
ba7         n = 1;
321         while (size > n) n <= 1;
7ce         t.resize(2*n);
62b     }

1cb     T op(T a, T b){
534         return a + b;
4ed     }

6ab     T query(int l, int r, int idx, int a, int b){
41a         if (r < a || b < l) return neutral;
877         if (l <= a && b <= r) return t[idx];
e26         int m = (a+b)/2;
0f2         return op(query(l, r, 2*idx, a, m), query(l, r,
fd6             2*idx+1, m+1, b));
190     T query(int l, int r){ return query(l, r, 1, 0, n
-1); }

671     void add(int idx, T v){
aab         if (idx < 0 || idx >= n) return;
cae         t[idx += n] += v; // aqui era pra ser op() invê
s de +=
4ef         for (idx >= 1; idx; idx >= 1) t[idx] = op(t
[2*idx], t[2*idx+1]);
7ef     }
fdf };
```

3 Grafos

3.1 Toposort

Retorna as componentes na ordenação topológica reversa. O número de componentes é $ncomp$. Talvez o certo seja $\min(low[u], num[v])$, mas vou usar assim até achar um erro.

Estado em $vis[]$:

- 0: não visitado
- 1: finalizado
- 2: ativo

```
e3b vector<int> g [mx], comp (mx), low (mx), num (mx), vis
(mx);
82b stack<int> vert;
22d int ti=0;
c02 int ncomp=0;
e0e void tarzan(int u){
873     low[u] = num[u] = ++ti;
10a     vis[u] = 2; // active
7a5     vert.push(u);

7b9     for (auto v: g[u]){
d90         if (!vis[v]){
5c1             tarzan(v);
ab6             low[u] = min(low[u], low[v]);
555         }
2d3         if (vis[v] == 2) low[u] = min(low[u], low[v]);
// talvez o certo seja min(low[u], num[v]) mas vou
usar assim até achar um erro
dd6     }

382     if (num[u] == low[u]){
a8f         ncomp++;
690         while (!vert.empty()){
08a             int v = vert.top(); vert.pop();
d6f             comp[v] = ncomp;
cca             vis[v] = 1;
163             if (u == v) break;
287         }
734     }
a7f }
```

4 Teoria e Apêndices

4.1 Permutations

- O código para calcular a paridade da permutação está no caderno.
- **Face Value:** Use as definições de permutação de forma literal. Uma permutação de tamanho N consiste em inteiros estritamente distintos de 1 a N .
- **Divisibilidade:** Devido à limitação de limite, existem no máximo $\lfloor N/k \rfloor$ elementos na permutação que são múltiplos de k .
- **Validação Rápida:** Como o multiset de uma permutação válida de tamanho M é sempre único $\{1, 2, \dots, M\}$, uma abordagem direta para checar se um subarray forma uma permutação é comparar os multisets utilizando algoritmos de **Hashing**.

4.2 Lis

- Encontrar a LIS de um array reverso é equivalente a encontrar a **LDS** (Longest Decreasing Subsequence).
- Necessita de adaptação para *longest non-increasing subsequence*. Para restaurar a subsequência, use um array auxiliar (vetor de pais).
- **Patience sorting:** Pensar no problema como formar pilhas, colocando o elemento atual na pilha com o menor topo que seja maior que ele. (Útil para modelagem geométrica ou de restrições).
- O número de LIS distintas pode ser calculado em $\mathcal{O}(N \log N)$ com SegTree.
- **LCS em Permutações:** A primeira permutação define o índice alvo para cada valor. Você substitui os valores da segunda permutação pelos índices correspondentes da primeira. A tarefa se reduz a achar a LIS do novo array, já que cada elemento aparece exatamente uma vez.